University of Amsterdam

Abstract

Temperature Modeling using (Stochastic) Differential Equations

11 februari 2026

Student:
Konstantinos Chatziandreou
k.chatziandreou@uva.nl
...

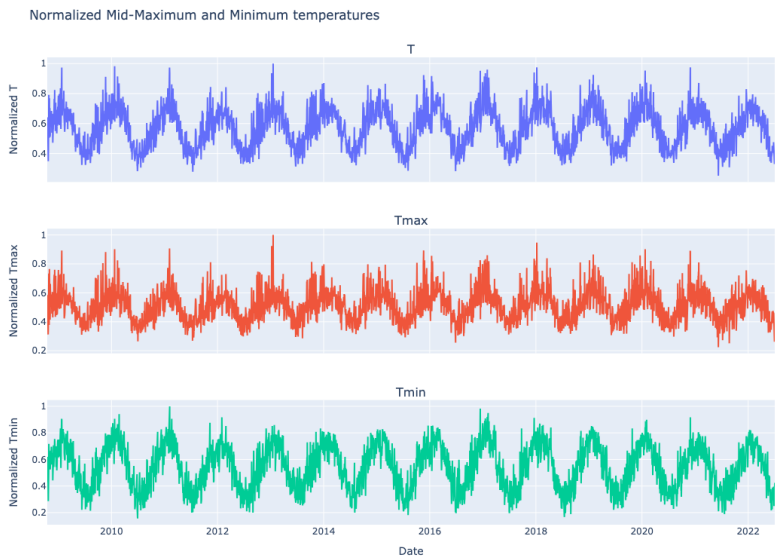
Academic Supervisor:
Drona Kandhai

1 Empirical Analysis of Temperature Dynamics

Before discussing the stochastic modeling of temperature we are first going to do a short empirical analysis on the temperature dynamics. As you might think temperature is a spatio-temporal process meaning that it varies both in space (for example across countries) but also in time. In what follows we will consider the modeling of the temperature in a specific location. Meaning that we use some measured temperature data from the National climatic data center. The data set we will consider for the following empirical analysis consist of four variables: daily maximum temperature ($Tmax$) in degrees (^oC) and minimum temperature ($Tmin$). We will use an approximation then of the average daily temperature by defining the variable

$$T = \frac{Tmin + Tmax}{2} \tag{1}$$

which will again be measured in degrees (^oC) and will denote the base temperature in our analysis. The following plot shows the evolution of these through variables in a span of approximately 12 years. We have also normalized our data using the maximum observed temperature.

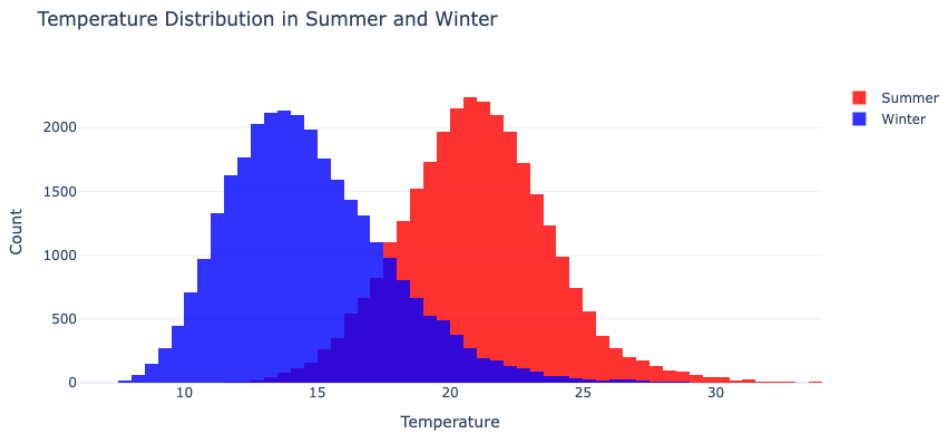


Figuur 1: Time series of daily Average, Maximum, Minimum temperatures

We can immediately see some first characteristics from these time series data. There is a clear

1507 (errors:4) words

mean-reverting effect taking place in the temperature dynamics. When consider a stochastic modeling for the description of these dynamics this will be something that we would like to take into consideration. Having a quick look on the distributions of the temperature across summer and and winter periods.



Figuur 2: Dsitribution of temperature for Summer and Winter periods

Let us look a bit closer on the seasonality of our time series by doing a time series decomposition. This time series decomposition method is a technique that splits our time series data into several components namely:

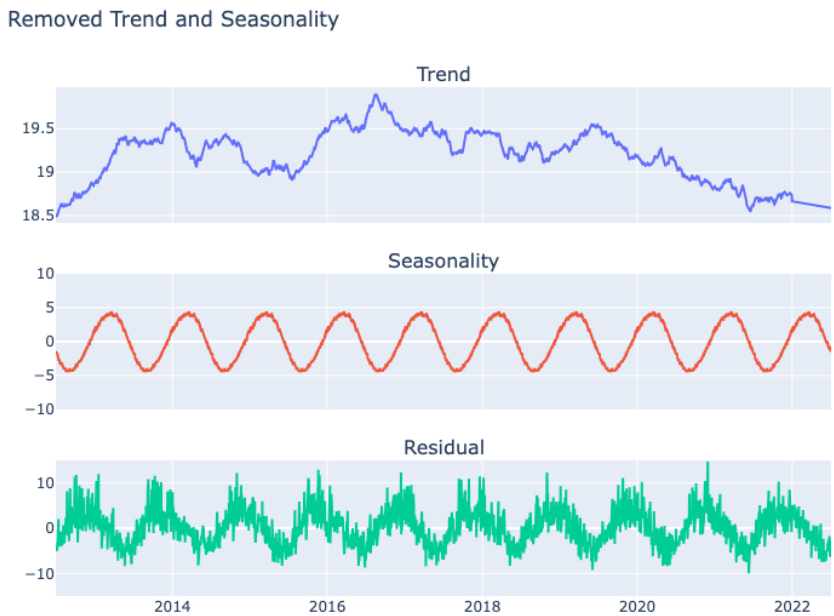
- Trend: decreasing, constant or increasing over time?
- Seasonality: What is the periodic signal describing the evolution of the time series data?
- Noise: Variability in the data that can not be explained in the model. This last component is very important because it is the one we are also going to use to model this random variability of the temperature dynamics using a Normally distributed noise.

The additive model then tries to decompose the temperature dynamics, which for now we are going to denote by y_t (when we are using the index $t \in [0, T]$ we will mean the process which evolves through time. This will in principle be also a randomly varying process. When we want to denote a process that only varies in a deterministic manner in time we will be denoting it by $y(t)$ so a deterministic function of time). This decomposition will then be written as

$$y_t = T_t + S(t) + \epsilon_t \tag{2}$$

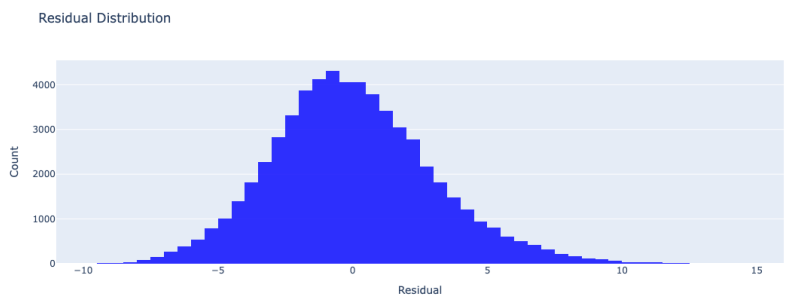
For a detailed analysis on these type of decompositions we also refer the following links in the statsmodels documentation of python Seasonal Decomposition. This is a naive decomposition Classical decomposition. The results are obtained by first estimating the trend by applying a convolution filter to the data. The trend is then removed from the series and the average of this de-trended series for each period is the returned seasonal component. The steps in classical decomposition are as follows:

- compute the trend-cycle component $\hat{T}_t$ using an m-MA (moving average).
- Calculate de-trended series $y_t - \hat{T}_t$. estimate seasonal component $\hat{S(t)}$, by averaging the detrended values for that season (in our case, this is each day of the year). Seasonal component values are adjusted to ensure they add to zero.
- the remainder is calculated $\epsilon_t = y_t - \hat{T}_t - \hat{S}_t$. This remainder will in general describe the "noiseïn our time series data.



Figuur 3: Additive decomposition of time series temperature data. The components of the decomposition denote the Trend (upwards/downwards drift in temperature dynamics), Seasonality and the residual error of the decomposition

After this decomposition we would like to examine the distribution of the residual errors to verify if they are approximately normally distributed. This will be an important assumption in what follows in the next Section and the modeling part. We can clearly observe that a good approximation of this error would be a normal distribution $\mathcal{N}(\mu, \sigma^2)$.



Figuur 4: Residual error distribution ϵ_t after the additive decomposition of the time series data

2 Stochastic Modeling of Temperature dynamics

Let us now discuss the modeling of the temperature dynamics. We will consider the modeling of this stochastic process using a stochastic differential equation. This differential equation essential describes the evolution of the temperature through time but also taking into account this random fluctuations that the temperature dynamics have in real life. We want these dynamics to capture to main characteristics of temperature data. Namely the mean-reverting property which essentially will describe the speed of mean-reversion of the temperature to its seasonal value and also the seasonality function $S(t)$ which for now will be denoted by $\bar{\mu}(t)$ and will describe the seasonal (deterministic) evolution of the temperature. To do so we consider the following stochastic differential equation which models the dynamics of the temperatures.

$$dT_t = \kappa\left(\bar{\mu}(t) - T_t\right)dt + \sigma(t)dW_t \tag{3}$$

where $W = (W_t)_{t\geq 0}$ is the driving "noise"which is taken to be a Winer process; α is the speed of mean-reversion and $\bar{\mu}(t)$ is the seasonal mean and $\sigma(t)$ the volatility of the process (which can also be a function of time) but we have to ensure that it is positive. You can just assume that it is a positive constant. To capture fully the mean-reverting dynamics of the temperature it is important to ensure that

$$\mathbb{E}\left[T_t\right] \approx \bar{\mu}(t) \tag{4}$$

The solution of this SDE can also be found using Ito's formula. I have a small reference in this appendix on this if you would like to delve more into this calculations A. Regarding the deterministic seasonal mean $\bar{\mu}(t)$, having a closer look at the "denoised"temperature time series it reveals that temperatures have uniform peaks and very weak rising trends. This inspires us to model the trend with a linear function and the seasonality with a single single sine function. Thus the determistic seasonal temperature which we have denoted by $\bar{\mu}(t)$ is modeled by

$$\bar{\mu}(t) = \alpha + \beta t + \gamma \sin\left(\omega t + \phi\right) \tag{5}$$

where the period $\omega = \frac{2\pi}{365}$ and the phase shift ϕ captures the fact that the minimum and maximum temperatures do not necessarily occur on the first of January or the first of July respectively. Using historical data we can estimate these parameters by employing a least-squares method. This can be done by observing that the equation (5) can be rewritten as

$$\bar{\mu}(t) = \alpha_1 + \alpha_2 + \alpha_3 sin(\omega t) + \alpha_4 cos(\omega t) \tag{6}$$

we find the vector $\xi = (\alpha_1, \alpha_2, \alpha_3, \alpha_4)$ that minimizes the estiamtion errors. Then

$$A = \alpha_1, \quad B = \alpha_2, \quad C = \sqrt{\alpha_3^2 + \alpha_4^2}, \quad \phi = tan^{-1}\left(\frac{\alpha_4}{\alpha_3}\right) - \pi$$

We refer to the following paper for some further analysis on this and some reasonable values of the parameter sets for the numerical simulations Temperature modeling.

2.1 Numerical Simulation of the SDE - Euler's Method

Exploring the solution of the SDE (3), as described analytically in Appendix A, we find that

$$T_t = \bar{\mu}(t) + (T_s - \bar{\mu}(s))e^{-\kappa(t-s)} + \int_s^t e^{-\kappa(t-u)}\sigma_t dW_u. \tag{7}$$

Hence, using a discretization period Δ (for example, $\Delta = 1$ day), we obtain the following Euler–Maruyama discretization of the solution of T_t:

$$T_{t+\Delta} - \bar{\mu}(t+\Delta) = e^{-\kappa\Delta}\left(T_t - \bar{\mu}(t)\right) + \sigma\int_t^{t+\Delta} e^{-\kappa(t+\Delta-\nu)}dW_u. \tag{8}$$

Since the stochastic integral

$$\int_t^{t+\Delta} e^{-\kappa(t+\Delta-\nu)}dW_\nu \tag{9}$$

is normally distributed with mean zero and variance

$$\mathbb{V}\left[\int_t^{t+\Delta} e^{-\kappa(t+\Delta-\nu)}dW_\nu\right] = \frac{1-e^{-2\kappa\Delta}}{2\kappa}, \tag{10}$$

we can approximate it using a normally distributed random variable $\xi \sim \mathcal{N}(0,1)$:

$$\int_t^{t+\Delta} e^{-\kappa(t+\Delta-\nu)}dW_\nu \approx \sqrt{\frac{1-e^{-2\kappa\Delta}}{2\kappa}}\,\xi. \tag{11}$$

Thus, the discrete-time evolution of T_t under Euler–Maruyama sampling is given by:

$$T_{t+\Delta} = \bar{\mu}(t+\Delta) + e^{-\kappa\Delta}(T_t - \bar{\mu}(t)) + \sigma\sqrt{\frac{1-e^{-2\kappa\Delta}}{2\kappa}}\,\xi. \tag{12}$$

The following algorithm summarizes the numerical implementation:

Algorithm 1 Euler–Maruyama Simulation of T_t

Require: κ, σ, $\bar{\mu}(t)$, T_0, Δ, N (number of time steps, such that $\Delta = t_N/N$)

```
Initialize T ← T_0
for n = 0, ..., N − 1 do
    Sample ξ_n ~ N(0, 1)
    Compute T_{n+1} using:
```

$$T_{n+1} = \bar{\mu}(t_n+\Delta) + e^{-\kappa\Delta}(T_n - \bar{\mu}(t_n)) + \sigma\sqrt{\frac{1-e^{-2\kappa\Delta}}{2\kappa}}\,\xi_n$$

```
end for
return {T_n}_{n=0}^N
```

2.2 Modeling Solar irradiance using SDEs

Some useful material for the modeling of solar irradiance can be found in the following papers:

- Day-ahead probabilistic forecast of solar irradiance: a Stochastic Differential Equation approach
-

A Using Itô's Formula

Using Itô calculus and solving the stochastic differential equation, we apply the Itô's formula in differential notation:

$$df(t,x) = f_t(t,x)dt + f_x(t,x)dx + \frac{1}{2}f_{xx}(t,x)dxdx, \tag{13}$$

where x is an Itô process. The challenge lies in selecting the function $f(t,x)$. We choose:

$$f(t,x) = e^{\kappa t}x, \tag{14}$$

which gives:

$$f_t(t,x) = \kappa e^{\kappa t}x, \quad f_x(t,x) = e^{\kappa t}, \quad f_{xx}(t,x) = 0. \tag{15}$$

This choice is motivated by the integrating factor method used in solving deterministic integrals.

B Solving the Stochastic Differential Equation

Applying the Itô-Doeblin formula to our SDE, where the Itô process is T_t with dynamics dT_t:

$$d(e^{\kappa t}T_t) = f_t(t,T_t)dt + f_x(t,T_t)dT_t + \frac{1}{2}f_{xx}(t,T_t)dT_tdT_t. \tag{16}$$

Substituting the expressions for f_t, f_x, and f_{xx}:

$$d(e^{\kappa t}T_t) = \kappa e^{\kappa t}T_t dt + e^{\kappa t}dT_t. \tag{17}$$

Using the given SDE dynamics:

$$d(e^{\kappa t}T_t) = \kappa e^{\kappa t}T_t dt + e^{\kappa t}\left[\kappa(\bar{\mu}(t) - T_t)dt + \sigma_t dW_t\right]. \tag{18}$$

Canceling the $\kappa e^{\kappa t}T_t dt$ terms:

$$d(e^{\kappa t}T_t) = \kappa e^{\kappa t}\bar{\mu}(t)dt + e^{\kappa t}\sigma_t dW_t. \tag{19}$$

Integrating both sides over $u \in [s, t]$:

$$\int_s^t d(e^{\kappa u}T_u) = e^{\kappa t}T_t - e^{\kappa s}T_s = \int_s^t \kappa e^{\kappa u}\bar{\mu}(u)du + \int_s^t e^{\kappa u}\sigma_t dW_u. \tag{20}$$

Rearranging:

$$e^{\kappa t}T_t = e^{\kappa s}T_s + \int_s^t \kappa e^{\kappa u}\bar{\mu}(u)du + \int_s^t e^{\kappa u}\sigma_t dW_u. \tag{21}$$

Changing the integral base to $d\bar{T}_u$:

$$e^{\kappa t}T_t = e^{\kappa s}T_s + \int_s^t e^{\kappa u}d\bar{\mu}(u) + \int_s^t e^{\kappa u}\sigma_t dW_u. \tag{22}$$

Dividing by $e^{\kappa t}$:

$$T_t = T_s e^{-\kappa(t-s)} + \int_s^t e^{-\kappa(t-u)}d\bar{\mu}(u) + \int_s^t e^{-\kappa(t-u)}\sigma_t dW_u. \tag{23}$$

Grouping terms:

$$T_t = \bar{T}_t + (T_s - \bar{\mu}(s))e^{-\kappa(t-s)} + \int_s^t e^{-\kappa(t-u)}\sigma_t dW_u. \tag{24}$$

This final result expresses the solution for T_t in terms of its mean-reverting behavior and stochastic perturbations.

B.1 Data for solar irradiance and temperatures

With the following code snippet you can download data for solar irradiance and temperatures. For a full documentation you can also refer to the following guthub API page Open meteo github API page.

Historical Data:

```
import openmeteo_requests
import requests_cache
import pandas as pd
from retry_requests import retry

# Setup the Open-Meteo API client with cache and retry on error
cache_session = requests_cache.CachedSession('.cache', expire_after=3600)
retry_session = retry(cache_session, retries=5, backoff_factor=0.2)
openmeteo = openmeteo_requests.Client(session=retry_session)

# Use the ERA5 archive endpoint for historical data
url = "https://archive-api.open-meteo.com/v1/era5"

# Adjust dates and variables as needed.
# Note that you cannot retrieve future dates in the archive endpoint.
params = {
    "latitude": 52.52,
    "longitude": 13.41,
    "start_date": "2020-08-10",
    "end_date": "2023-08-23",
    "hourly": "temperature_2m",
    "daily": "temperature_2m_mean"
}

# Request data
responses = openmeteo.weather_api(url, params=params)
```

```
# Process first location. Add a for-loop if you need multiple locations/models
response = responses[0]
print(f"Elevation: {response.Elevation()} m")
print(f"Timezone: {response.Timezone()} {response.TimezoneAbbreviation()}")
print(f"UTC Offset (s): {response.UtcOffsetSeconds()}")

# Process hourly data
hourly = response.Hourly()
hourly_temp = hourly.Variables(0).ValuesAsNumpy()

hourly_data = {
    "date": pd.date_range(
        start=pd.to_datetime(hourly.Time(), unit="s", utc=True),
        end=pd.to_datetime(hourly.TimeEnd(), unit="s", utc=True),
        freq=pd.Timedelta(seconds=hourly.Interval()),
        inclusive="left"
    ),
    "temperature_2m": hourly_temp
}

hourly_dataframe = pd.DataFrame(data=hourly_data)
print("\nHourly data:")
print(hourly_dataframe)

# Process daily data
daily = response.Daily()
daily_temp_mean = daily.Variables(0).ValuesAsNumpy()

daily_data = {
    "date": pd.date_range(
        start=pd.to_datetime(daily.Time(), unit="s", utc=True),
        end=pd.to_datetime(daily.TimeEnd(), unit="s", utc=True),
        freq=pd.Timedelta(seconds=daily.Interval()),
        inclusive="left"
    ),
    "temperature_2m_mean": daily_temp_mean
}

daily_dataframe = pd.DataFrame(data=daily_data)
print("\nDaily data:")
print(daily_dataframe)


# solar irradiance data
import openmeteo_requests
import requests_cache
import pandas as pd
from retry_requests import retry

# Setup the Open-Meteo API client with cache and retry on error
cache_session = requests_cache.CachedSession('.cache', expire_after=3600)
retry_session = retry(cache_session, retries=5, backoff_factor=0.2)
openmeteo = openmeteo_requests.Client(session=retry_session)

# Use the ERA5 archive endpoint for historical data
url = "https://archive-api.open-meteo.com/v1/era5"
```

```
params = {
    "latitude": 52.52,
    "longitude": 13.41,
    "start_date": "2020-08-10",
    "end_date": "2023-08-23",
    # Request solar irradiance data
    "hourly": "shortwave_radiation",
    "daily": "shortwave_radiation_sum"
}

# Request data
responses = openmeteo.weather_api(url, params=params)

# Process first location. Add a loop if multiple locations are needed
response = responses[0]
print(f"Elevation: {response.Elevation()} m")
print(f"Timezone: {response.Timezone()} {response.TimezoneAbbreviation()}")
print(f"UTC Offset (s): {response.UtcOffsetSeconds()}")

# Process hourly data
hourly = response.Hourly()
# The 0th variable in "hourly" is the shortwave_radiation
hourly_shortwave = hourly.Variables(0).ValuesAsNumpy()

hourly_data = {
    "date": pd.date_range(
        start=pd.to_datetime(hourly.Time(), unit="s", utc=True),
        end=pd.to_datetime(hourly.TimeEnd(), unit="s", utc=True),
        freq=pd.Timedelta(seconds=hourly.Interval()),
        inclusive="left"
    ),
    "shortwave_radiation": hourly_shortwave
}

hourly_dataframe = pd.DataFrame(data=hourly_data)
print("\nHourly shortwave radiation data:")
print(hourly_dataframe)

# Process daily data
daily = response.Daily()
# The 0th variable in "daily" is shortwave_radiation_sum
daily_shortwave_sum = daily.Variables(0).ValuesAsNumpy()

daily_data = {
    "date": pd.date_range(
        start=pd.to_datetime(daily.Time(), unit="s", utc=True),
        end=pd.to_datetime(daily.TimeEnd(), unit="s", utc=True),
        freq=pd.Timedelta(seconds=daily.Interval()),
        inclusive="left"
    ),
    "shortwave_radiation_sum": daily_shortwave_sum
}

daily_dataframe = pd.DataFrame(data=daily_data)
print("\nDaily shortwave radiation sum data:")
print(daily_dataframe)
```

Forecast data:

```
import openmeteo_requests
```

```
import requests_cache
import pandas as pd
from retry_requests import retry

# Setup the Open-Meteo API client with cache and retry on error
cache_session = requests_cache.CachedSession('.cache', expire_after = 3600)
retry_session = retry(cache_session, retries = 5, backoff_factor = 0.2)
openmeteo = openmeteo_requests.Client(session = retry_session)

# Make sure all required weather variables are listed here
# The order of variables in hourly or daily is important to assign them correctly below
url = "https://historical-forecast-api.open-meteo.com/v1/forecast"
params = {
    # Berlin area (you can change )
        "latitude": 52.52,
        "longitude": 13.41,
        "start_date": "2020-08-10",
        "end_date": "2024-08-23",
        "hourly": "temperature_2m",
        "daily": "temperature_2m_mean"
}
responses = openmeteo.weather_api(url, params=params)

# Process first location. Add a for-loop for multiple locations or weather models
response = responses[0]
print(f"Elevation {response.Elevation()} m asl")
print(f"Timezone {response.Timezone()} {response.TimezoneAbbreviation()}")
print(f"Timezone difference to GMT+0 {response.UtcOffsetSeconds()} s")

# Process hourly data. The order of variables needs to be the same as requested.
hourly = response.Hourly()
hourly_shortwave_radiation = hourly.Variables(0).ValuesAsNumpy()

hourly_data = {"date": pd.date_range(
        start = pd.to_datetime(hourly.Time(), unit = "s", utc = True),
        end = pd.to_datetime(hourly.TimeEnd(), unit = "s", utc = True),
        freq = pd.Timedelta(seconds = hourly.Interval()),
        inclusive = "left"
)}
hourly_data["temperature_2m"] = hourly_shortwave_radiation

hourly_dataframe = pd.DataFrame(data = hourly_data)
print(hourly_dataframe)

# Process daily data. The order of variables needs to be the same as requested.
daily = response.Daily()
daily_shortwave_radiation_sum = daily.Variables(0).ValuesAsNumpy()

daily_data = {"date": pd.date_range(
        start = pd.to_datetime(daily.Time(), unit = "s", utc = True),
        end = pd.to_datetime(daily.TimeEnd(), unit = "s", utc = True),
        freq = pd.Timedelta(seconds = daily.Interval()),
        inclusive = "left"
)}
daily_data["temperature_2m_sum"] = daily_shortwave_radiation_sum

daily_dataframe = pd.DataFrame(data = daily_data)
```

```
print(daily_dataframe)


#solar
import openmeteo_requests
import requests_cache
import pandas as pd
from retry_requests import retry

# Setup the Open-Meteo API client with cache and retry on error
cache_session = requests_cache.CachedSession('.cache', expire_after = 3600)
retry_session = retry(cache_session, retries = 5, backoff_factor = 0.2)
openmeteo = openmeteo_requests.Client(session = retry_session)

# Make sure all required weather variables are listed here
# The order of variables in hourly or daily is important to assign them correctly below
url = "https://historical-forecast-api.open-meteo.com/v1/forecast"
params = {
        "latitude": 52.52,
        "longitude": 13.41,
        "start_date": "2024-08-10",
        "end_date": "2024-08-23",
        "hourly": "shortwave_radiation",
        "daily": "shortwave_radiation_sum"
}
responses = openmeteo.weather_api(url, params=params)

# Process first location. Add a for-loop for multiple locations or weather models
response = responses[0]

print(f"Elevation {response.Elevation()} m asl")
print(f"Timezone {response.Timezone()} {response.TimezoneAbbreviation()}")
print(f"Timezone difference to GMT+0 {response.UtcOffsetSeconds()} s")

# Process hourly data. The order of variables needs to be the same as requested.
hourly = response.Hourly()
hourly_shortwave_radiation = hourly.Variables(0).ValuesAsNumpy()

hourly_data = {"date": pd.date_range(
        start = pd.to_datetime(hourly.Time(), unit = "s", utc = True),
        end = pd.to_datetime(hourly.TimeEnd(), unit = "s", utc = True),
        freq = pd.Timedelta(seconds = hourly.Interval()),
        inclusive = "left"
)}
hourly_data["shortwave_radiation"] = hourly_shortwave_radiation

hourly_dataframe = pd.DataFrame(data = hourly_data)
print(hourly_dataframe)

# Process daily data. The order of variables needs to be the same as requested.
daily = response.Daily()
daily_shortwave_radiation_sum = daily.Variables(0).ValuesAsNumpy()

daily_data = {"date": pd.date_range(
        start = pd.to_datetime(daily.Time(), unit = "s", utc = True),
        end = pd.to_datetime(daily.TimeEnd(), unit = "s", utc = True),
        freq = pd.Timedelta(seconds = daily.Interval()),
        inclusive = "left"
```

```
)}
daily_data["shortwave_radiation_sum"] = daily_shortwave_radiation_sum

daily_dataframe = pd.DataFrame(data = daily_data)
print(daily_dataframe)
```

B.2 Data for power production (Solar and Wind)

You can also download European energy data through the ENTSO-E API. This will however provide you only realized power production for wind (onshore and offshore) as well as solar power production. Typically such production are found based on production curves which transform wind speeds and solar irradiance to actual production measurements which are given in MWh. For a detailed description of the API package you can see the following reference Entsoe API github page. To download these type of data you have to create an API key which is described in the following page API KEY Configuration.

```
import pandas as pd
import plotly.graph_objects as go
from plotly.subplots import make_subplots
from entsoe import EntsoePandasClient
from datetime import datetime, timedelta


# here you have to generate a new api_key from

# https://transparency.entsoe.eu/content/static_content/download?path=/Static%20content/API-Tok
client = EntsoePandasClient(api_key='ac01d108-57af-480f-96b3-b2a44de9c797')

# Define the time range
start = pd.Timestamp(datetime.now() - timedelta(days=365), tz='Europe/Brussels')
end = pd.Timestamp(datetime.now(), tz='Europe/Brussels')


# Define the country code (example: 'DE' for Germany)
country_code = 'DE_LU'
day_ahead_prices =  client.query_day_ahead_prices(country_code, start = start, end = end)

# Fetch data for solar, onshore wind, and offshore wind production
solar = client.query_generation(country_code, start=start, end=end, psr_type='B16')
onshore_wind = client.query_generation(country_code, start=start, end=end, psr_type='B19')
offshore_wind = client.query_generation(country_code, start=start, end=end, psr_type='B18') # C


# Normalize the data
solar_norm = solar / solar.max()
onshore_wind_norm = onshore_wind / onshore_wind.max()
offshore_wind_norm = offshore_wind / offshore_wind.max()
```